

JWT

RBAC

ASP.NET 8

Visual

Авторизация и RBAC в веб-сервисах

JSON Web Tokens • Role-Based Access Control • [Authorize]

Проблема — API без охраны

Представим API интернет-магазина. Все эндпоинты открыты любому, у кого есть интернет.

GET /api/product/getAll

Смотри каталог



ОК — публичные данные

POST /api/product

Добавить товар



Любой добавит спам

DELETE /api/product/5

Удалить товар



Любой обнулит склад

DELETE /api/users/all

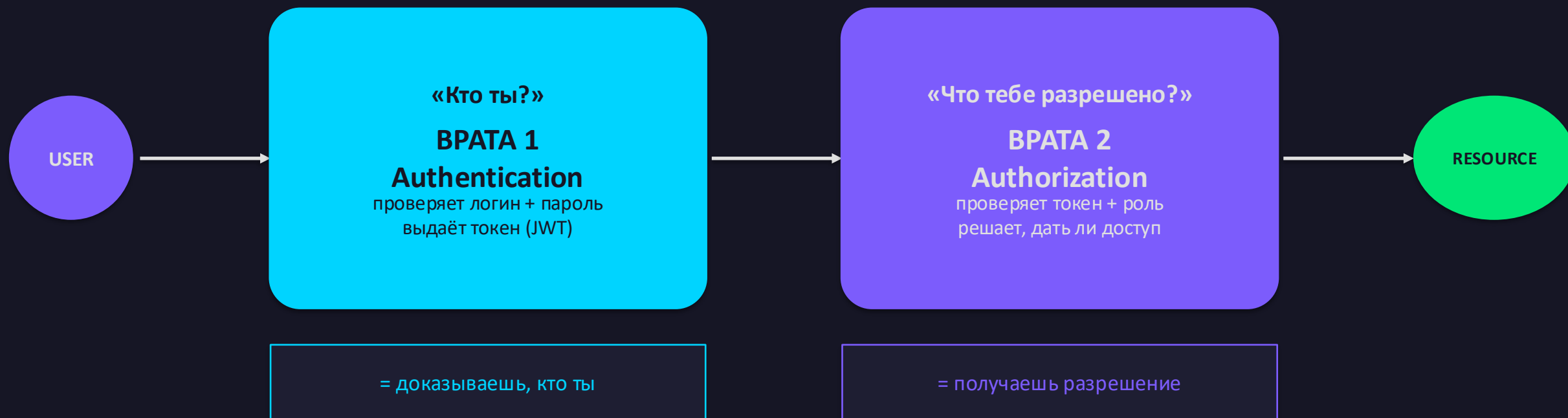
Удалить всех пользователей



Катастрофа

Вывод: нужен механизм, который (1) идентифицирует пользователя и (2) проверяет, что ему разрешено.

Authentication vs Authorization — двое ворот



Аналогия: концерт

Врата 1 = показываешь паспорт на входе ("да, ты Иван") → получаешь цветной браслет

Врата 2 = браслет открывает разные зоны (VIP, Стоячие, Бэкстейдж). Красный браслет ≠ доступ за кулисы.

Почему JWT? Stateful vs Stateless

SERVER

Sessions DB

- sess_a3f → user 1
- sess_b8z → user 2
- sess_c4q → user 3

Проблемы:

КЛАССИЧЕСКАЯ МОДЕЛЬ — сессии на сервере

- ✗ Сервер хранит память по каждому пользователю
- ✗ Горизонтальное масштабирование = сложно (sticky sessions)
- ✗ Рестарт сервера = все пользователи разлогинены
- ✗ Cross-domain = сложно

SERVER

- ✓ проверяет signature
- ✓ проверяет срок
(без обращения к БД)

Преимущества:

МОДЕЛЬ JWT — Stateless

- ✓ Токен несёт всё, что нужно
- ✓ Масштабирование — любой инстанс может валидировать
- ✓ Рестарт не страшен — токен остаётся валиден
- ✓ Cross-domain — один токен, много API

Визуальная анатомия JWT



Header и Payload — это просто Base64, НЕ шифрование!

Как выглядит в токене:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
```

decode

Что внутри на самом деле:

```
{ "alg": "HS256", "typ": "JWT" }
```

Payload encoded:

```
eyJzdWIiOiI0MiIsIm5hbWUiOiJpb25lbCI6ImV4cCI6MTcwMDAwMzYwMH0
```

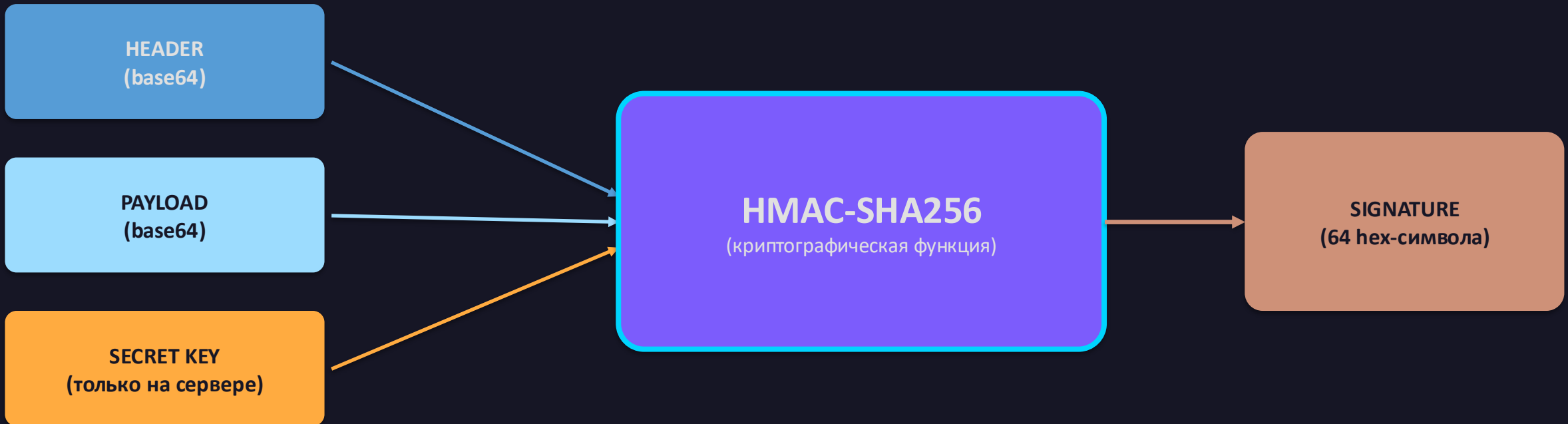
Payload decoded:

```
{  
  "sub": "42",  
  "name": "ivan",  
  "role": "Admin",  
  "exp": 1700003600  
}
```

⚠ ЛЮБОЙ может декодировать Header и Payload — это просто Base64. Браузер / JS делают это в 2 строки.

Вывод: НЕ КЛАДЁМ чувствительные данные в Payload (пароли, паспорт, IBAN, секреты). Только идентификаторы и роли.

Signature — как вычисляется (HMAC-SHA256)



Детерминированный

Те же входы → всегда тот же signature.
Изменили запятую → совершенно другой signature.

Однонаправленный

Из signature НЕЛЬЗЯ восстановить секретный ключ.
(В отличие от шифрования — расшифровки здесь нет.)

Что будет, если токен изменят?

Сценарий 1 — оригинальный токен



Сервер: $\text{HMAC}(\text{Header.Payload}, \text{secret}) = \text{Sig_X} \rightarrow \text{совпадение} \rightarrow \text{принят}$

Сценарий 2 — атакующий меняет роль "User" → "Admin" (но НЕ знает секрет)



Сервер: $\text{HMAC}(\text{Header.Payload_изменён}, \text{secret}) = \text{Sig_Y} \neq \text{Sig_X} \rightarrow 401 \text{ отклонён}$

Сценарий 3 — атакующий пытается пересчитать signature (без секрета)



HMAC требует секрет. У атакующего его нет. Brute-force HS256 с ключом 32+ символов — практически невозможно.

Вот почему JWT БЕЗОПАСЕН: signature связывает содержимое с секретом — меняешь содержимое, signature перестает совпадать.

Bearer-токен — «кто его держит, тот и использует»

TOKEN: eyJhbGciOiJIUzI1Ni...

Владелец: user 42 • Роль: Admin • Действителен до 12:00

«Bearer» = носитель, тот, кто его предъявляет

Как передаётся

Authorization: Bearer <token>

в HTTP-заголовке, при каждом запросе

Внимание

Кто украл токен → получил доступ.

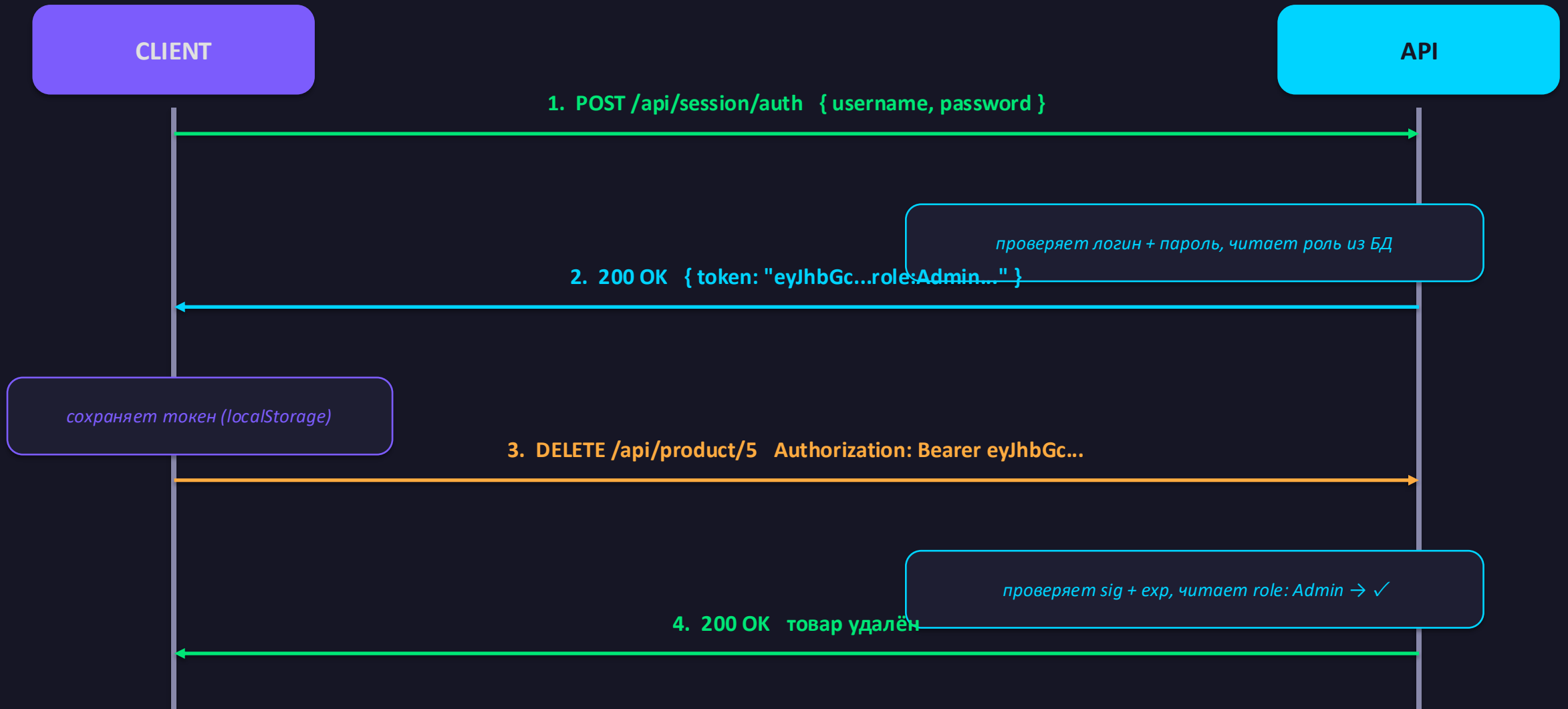
В проде ТОЛЬКО HTTPS.

Защита

Короткое время жизни (15–60 мин)

Refresh tokens для длинных сессий

Полный поток — Login → защищённый запрос



Жизненный цикл токена

Токен — как билет с напечатанной датой окончания. Сервер проверяет "exp" при каждом запросе.



Claim "iat" — issued at

Unix timestamp: когда токен был выдан.

Полезно для аудита («этот пользователь делал логин в 12:00»).

Claim "exp" — expiration

Unix timestamp: после него токен невалиден.

ASP.NET Core проверит автоматически, если поставить `ValidateLifetime = true`.

Настройка — пакеты и конфигурация

1. NuGet-пакет:

```
dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer
```

2. appsettings.json — добавляем секцию "Jwt":

```
{
  "ConnectionStrings": { ... },
  "Jwt": {
    "Issuer": "eUShopApi",           // кто выдаёт
    "Audience": "eUShopClients",    // для кого
    "Key": "tw_curs2026_super_secret_min_32_caractere!", // секрет
    "ExpireMinutes": 60
  }
}
```

Issuer

Кто выдаёт токен.
(наш API)

Audience

Кому предназначен.
(клиенты API)

Key

Секрет для signature.
Минимум 32 символа.

ExpireMinutes

Сколько действителен.
(15–60 минут)

TokenService — генерация JWT из Claims

```
public class TokenService
{
    private const string Issuer = "eUShopApi";
    private const string Audience = "eUShopClients";
    private const string SecretKey = "tw_curs2026_super_secret_min_32_caractere!";

    public string GenerateToken(int userId, string userName, string role)
    {
        // 1. Симметричный ключ из секрета
        var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(SecretKey));
        var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

        // 2. Claims = данные, которые кладём в payload
        var claims = new[] {
            new Claim(ClaimTypes.NameIdentifier, userId.ToString()),
            new Claim(ClaimTypes.Name, userName),
            new Claim(ClaimTypes.Role, role)           // <-- роль попадает в токен (RBAC)
        };

        // 3. Собираем токен
        var token = new JwtSecurityToken(
            issuer: Issuer, audience: Audience, claims: claims,
            expires: DateTime.UtcNow.AddMinutes(60), signingCredentials: creds);

        // 4. Сериализуем в Header.Payload.Signature
        return new JwtSecurityTokenHandler().WriteToken(token);
    }
}
```

Program.cs — настройка middleware

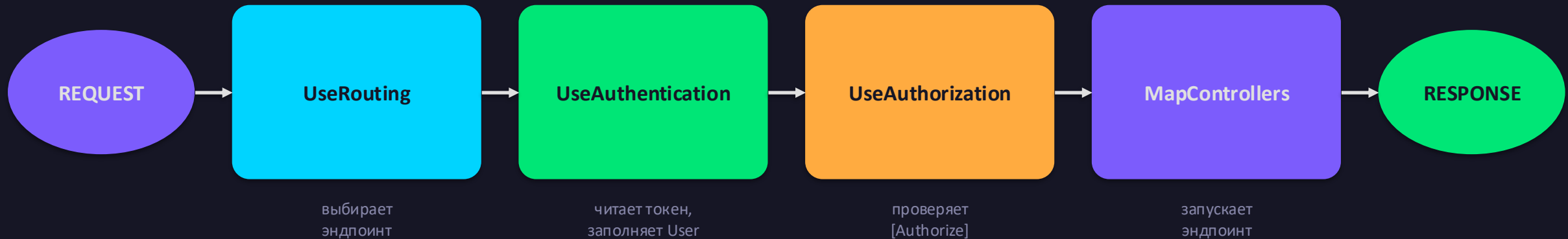
```
var builder = WebApplication.CreateBuilder(args);

// 1. Регистрируем аутентификацию по схеме JWT Bearer
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = new TokenValidationParameters {
            ValidateIssuer = true, ValidIssuer = "eUShopApi",
            ValidateAudience = true, ValidAudience = "eUShopClients",
            ValidateLifetime = true,                // проверка "exp"
            ValidateIssuerSigningKey = true,
            IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(
                "tw_curs2026_super_secret_min_32_caractere!"))
        };
    });
builder.Services.AddAuthorization();

var app = builder.Build();

// 2. Порядок важен! Authentication ПЕРЕД Authorization
app.UseAuthentication(); // читает токен, заполняет User
app.UseAuthorization();  // проверяет [Authorize] и роли
app.MapControllers();
app.Run();
```

Middleware Pipeline — как проходит запрос



Pipeline может прерваться на любом этапе:

UseAuthentication не прошёл

- Токена нет
- Токен с неверным signature
- Токен истёк

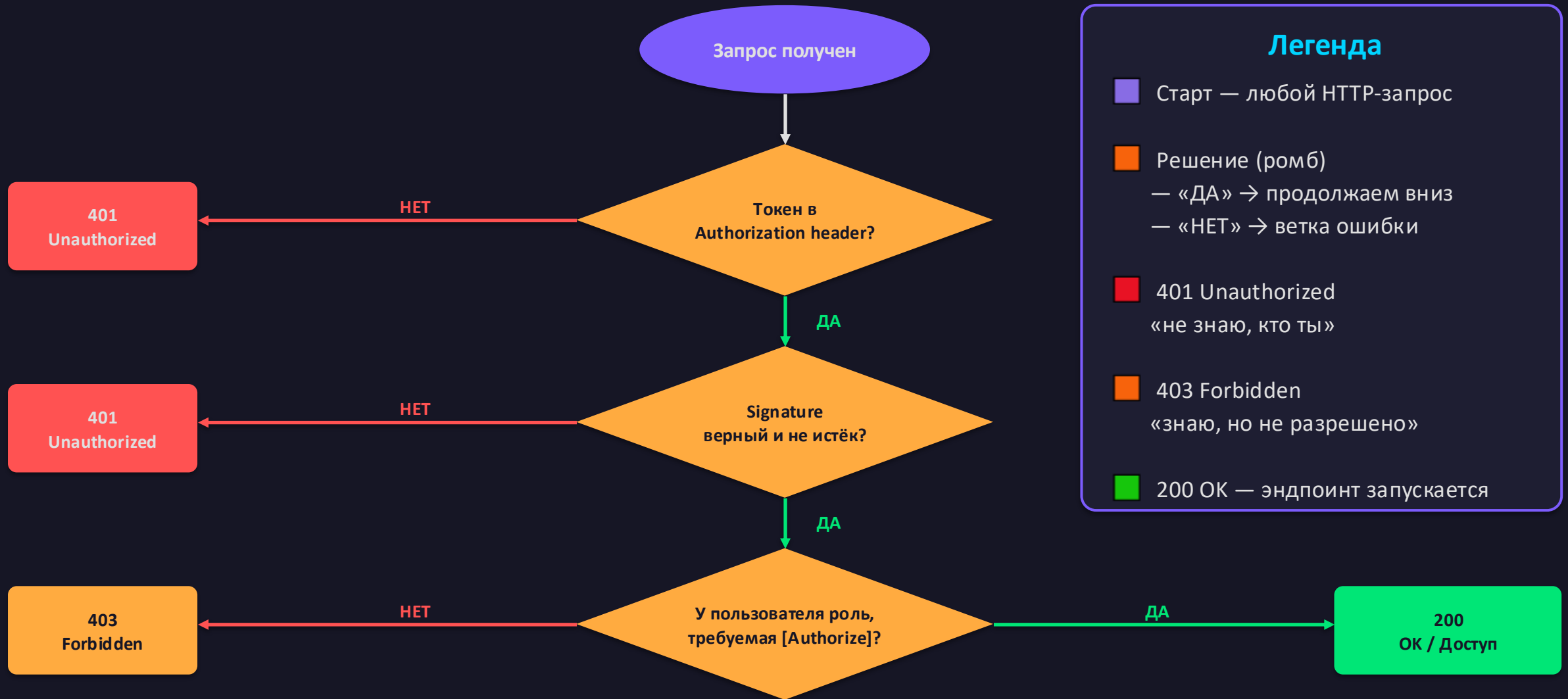
→ Pipeline ОСТАНАВЛИВАЕТСЯ. Ответ: 401 Unauthorized

UseAuthorization не прошёл

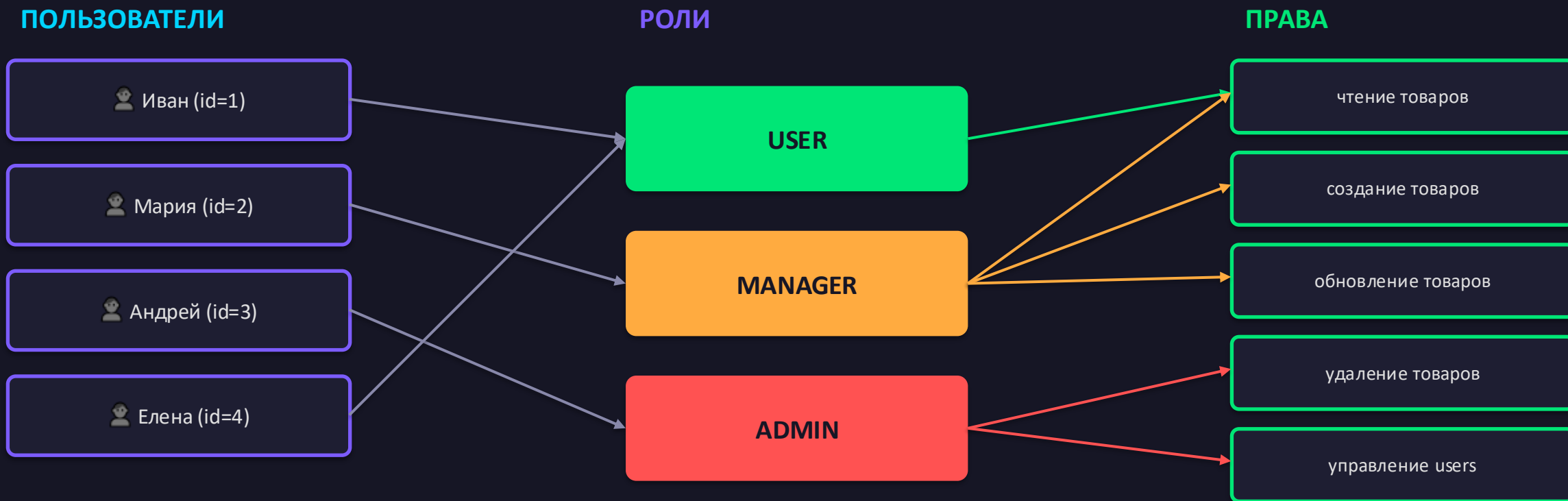
- Токен валиден, НО
- У пользователя нет роли, требуемой [Authorize(Roles=...)]

→ Pipeline ОСТАНАВЛИВАЕТСЯ. Ответ: 403 Forbidden

Как решает [Authorize] — дерево решений

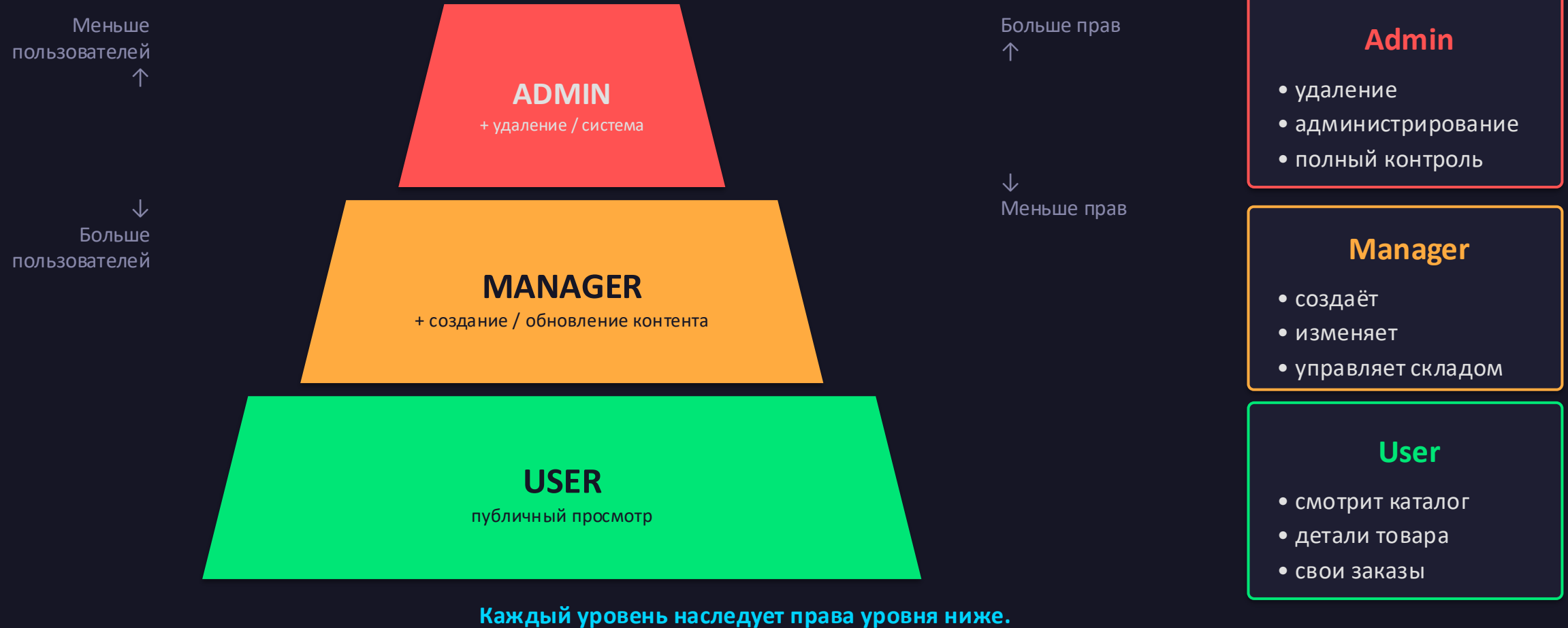


RBAC — Role-Based Access Control



RBAC = пользователям выдаются РОЛИ, у ролей — ПРАВА. Меняешь роль → мгновенно меняется всё, что доступно пользователю.

Иерархия ролей — пирамида прав



Матрица прав — кто, что, где

Действие	Public	User	Manager	Admin
 GET /products	✓	✓	✓	✓
 GET /products/{id}	✓	✓	✓	✓
 POST /products	✗	✗	✓	✓
 PUT /products/{id}	✗	✗	✓	✓
 DELETE /products/{id}	✗	✗	✗	✓
 GET /categories	✓	✓	✓	✓
 POST /categories	✗	✗	✓	✓
 DELETE /categories/{id}	✗	✗	✗	✓
 /admin/users (manage)	✗	✗	✗	✓

Мнемоника: читаешь = public • изменяешь = Manager+ • удаляешь / администрируешь = только Admin.

Как роль попадает в токен? Claim «role»

Payload JWT — claims:

```
{  
  "sub": "42",  
  "unique_name": "ivan",  
  "role": "Admin",  
  "iss": "eUShopApi",  
  "aud": "eUShopClients",  
  "iat": 1700000000,  
  "exp": 170003600  
}
```

ASP.NET Core автоматически мапит

claim «role» → User.IsInRole(...)
→ [Authorize(Roles = "Admin")]
работает из коробки

В TokenService:

```
new Claim(  
    ClaimTypes.Role,  
    user.Role.ToString()  
)
```

UserRole (enum из Domain):

```
public enum UserRole { User = 1, Manager = 20, Admin = 30 }
```

Синтаксис [Authorize] — варианты

```
[Authorize] // любой аутентифицированный пользователь

[Authorize(Roles = "Admin")] // строго — только Admin

[Authorize(Roles = "Admin,Manager")] // ИЛИ — Admin или Manager

[Authorize(Roles = "Admin")]
[Authorize(Roles = "Manager")] // AND — И Admin И Manager
// (редко; можно иметь 2 роли одновременно)

[AllowAnonymous] // override — отменяет [Authorize]
```

✓ На контроллере (класс)

Применяется ко ВСЕМ действиям.
Исключение: те, что с [AllowAnonymous].

Удобно, когда большинство эндпоинтов
контроллера имеют одинаковые требования.

✓ На действии (метод)

Применяется только к этому методу.
Перекрывает правило класса.

Удобно, когда разные эндпоинты
одного контроллера имеют разные правила.

Стратегия применения в проекте

Endpoint	Public	User	Manager	Admin	Атрибут
GET /api/health/status	ДА	—	—	—	[AllowAnonymous]
GET /api/session/status	ДА	—	—	—	[AllowAnonymous]
POST /api/session/auth	ДА	—	—	—	[AllowAnonymous]
GET /api/product/getAll	ДА	—	—	—	[AllowAnonymous]
GET /api/product/{id}	ДА	—	—	—	[AllowAnonymous]
POST /api/product	НЕТ	НЕТ	ДА	ДА	[Authorize(Roles="Admin,Manager")]
PUT /api/product	НЕТ	НЕТ	ДА	ДА	[Authorize(Roles="Admin,Manager")]
DELETE /api/product/{id}	НЕТ	НЕТ	НЕТ	ДА	[Authorize(Roles="Admin")]
GET /api/order	НЕТ	НЕТ	ДА	ДА	[Authorize(Roles="Admin,Manager")]
GET /api/order/{id}	НЕТ	ДА	ДА	ДА	[Authorize]
POST /api/order	НЕТ	ДА	ДА	ДА	[Authorize]
PUT /api/order	НЕТ	НЕТ	ДА	ДА	[Authorize(Roles="Admin,Manager")]
DELETE /api/order/{id}	НЕТ	НЕТ	НЕТ	ДА	[Authorize(Roles="Admin")]

Читаешь = public. Изменяешь = Manager+. Удаляешь / администрируешь = только Admin.

Реальный код — ProductController с RBAC

```
[Route("api/product")]
[ApiController]
[Authorize] // по умолчанию: нужна аутентификация
public class ProductController : ControllerBase
{
    [HttpGet]
    [AllowAnonymous] // PUBLIC — все видят каталог
    public IActionResult GetAll() { ... }

    [HttpGet("{id}")]
    [AllowAnonymous] // PUBLIC — увидеть товар
    public IActionResult GetById(int id) { ... }

    [HttpPost]
    [Authorize(Roles = "Admin,Manager")] // Manager+ могут добавлять
    public IActionResult Create([FromBody] ProductDto p) { ... }

    [HttpPut("{id}")]
    [Authorize(Roles = "Admin,Manager")] // Manager+ могут редактировать
    public IActionResult Update(int id, [FromBody] ProductDto p) { ... }

    [HttpDelete("{id}")]
    [Authorize(Roles = "Admin")] // строго — только Admin
    public IActionResult Delete(int id) { ... }
}
```

Тестирование в Swagger / Postman

Шаг 1 — логин для получения токена:

```
POST /api/session/auth

{
  "login": "user",
  "password": "qwerty"
}
```

Ответ 200 OK:

```
{
  "token": "eyJhbGciOiJIUzI1NiI..."
```

Шаг 2 — отправь запрос с Authorization-заголовком:

```
DELETE /api/product/5

Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Шаг 3 — возможные ответы:

200 OK
product deleted

(твоя роль — Admin)

403 Forbidden
токен есть, но нет прав

(твоя роль — User)

401 Unauthorized
токен неверный или отсутствует

(забыл заголовок)



В Swagger UI: кнопка « Authorize» в правом верхнем углу → введи «Bearer <токен>» → все запросы автоматически получат заголовок.

Частые ошибки — и как их избежать

✗ 1. Секретный ключ в исходниках, закоммичен в Git

→ Любой, кто клонирует репозиторий, сможет подписывать токены. Локально — User Secrets, в проде — env-переменные.

✗ 2. `app.UseAuthentication()` отсутствует или стоит после `UseAuthorization()`

→ Порядок важен: СНАЧАЛА `UseAuthentication()` (читает токен), ПОТОМ `UseAuthorization()` (проверяет права). Иначе User всегда анонимен.

✗ 3. Чувствительные данные в payload

→ Payload — это просто Base64, ЛЮБОЙ может его прочесть. Не кладите пароли, паспорт, IBAN, секреты. Только идентификаторы и роли.

✗ 4. Токен без срока (или на 30 дней)

→ Украденный токен = доступ на 30 дней. Ставьте "expires" 15–60 минут. Для длинных сессий → refresh tokens.

✗ 5. `[Authorize]` на контроллере без `[AllowAnonymous]` на логине

→ Если логин требует `[Authorize]`, никто не сможет залогиниться — классическая «курица и яйцо».

✗ 6. Проверка роли в коде через `if (User.IsInRole(...))`

→ Избегайте разбросанных if. Используйте `[Authorize(Roles=...)]` декларативно — легче читать и аудить.

Best practices — проверенный список



Безопасность

- ✓ Секрет в User Secrets / env-переменных
НИКОГДА в исходниках
- ✓ Длина ключа: минимум 32 символа
для HS256
- ✓ Проверяйте ВСЁ: Issuer, Audience,
Lifetime, Signing Key
- ✓ В проде обязательно HTTPS
(токен в транзите — как пароль)
- ✓ Срок токена: 15–60 минут
для длинных сессий → refresh tokens
- ✓ НЕ кладите чувствительные данные в payload



Дизайн RBAC

- ✓ RBAC декларативно — атрибуты, НЕ if
[Authorize(Roles="Admin")]
- ✓ Default deny — [Authorize] на классе,
[AllowAnonymous] только где нужно
- ✓ Чёткие роли: User / Manager / Admin
не слишком много (макс. 5–7)
- ✓ Права в матрице — задокументированы
и сверяются при каждом новом эндпоинте
- ✓ Logout = клиент удаляет токен
(server stateless)
- ✓ Audit log для чувствительных действий